marinedustrie.de → Skipily: Shopify integration via Make.com

Step-by-step guide. With Make AI: the AI scenario assistant builds the skeleton, an AI text module processes each product (shorten name, remove HTML, suggest category). Status: product sync via x-api-key, load-hardened with rate limit.

---

1. Overview (two directions)

- **Products:** Shopify (marinedustrie.de) → Make → Skipily shop (automatic on create/update).
- **Orders:** Skipily → Make → Shopify (create order/note). Optional, described at the end.

2. What you need

- Skipily **provider key** (provider portal, Pro plan) for the x-api-key header.
- Shopify admin of marinedustrie.de (for webhooks).
- A Make.com account. For the AI text module, AI access in Make (Make AI or a connected OpenAI/Claude account).

3. Flow A: products to Skipily (step by step)

Step 1: Create the scenario with Make AI

In Make: **Create a new scenario** → use the AI assistant at the top („*Create with AI*“ / prompt field) and paste the following text. The AI creates trigger, AI module and HTTP module as a skeleton:

AI (Make): Build a Make scenario: Trigger = Custom Webhook (receives Shopify products/create and products/update). Then an AI text module that shortens the product name, strips HTML from the description and suggests a top category. Finally an HTTP module: POST JSON to a Skipily API with header x-api-key.

The AI creates a draft. You set the details (URL, header, mapping) in the next steps.

Step 2: Trigger = Custom Webhook

- First module **Webhooks > Custom webhook** → **Add** → **copy the URL** it generates.

Step 3: Shopify webhooks to the Make URL

- Shopify admin: **Settings** → **Notifications** → **Webhooks**.
- Create two webhooks (format **JSON**) to the Make URL: **Product creation** (products/create) and **Product update** (products/update).

Note: To test, save a product in Shopify. In Make, enable „Run once“ so the webhook learns the data structure.

Step 4: AI module - process product data

Module **AI text** (Make AI / OpenAI / Claude). It receives the Shopify product and returns clean JSON (short name, description without HTML, matching category). This turns the often long marinedustrie titles into a clean shop entry.

AI (Make): You receive a Shopify product (title, vendor, body_html, product_type, tags). Return ONLY JSON: {"name": short clear product name without marketing fluff, "description": description as plain text without HTML (max 400 chars), "category": one of the Skipily categories that fits best}. Choose the category from this list: .

Note: Fetch the Skipily category names once: GET .../rest/v1/product_categories?select=name_de,slug (header apikey: anon). Paste that list into the prompt. Without mapping, category_id stays null (product works, just uncategorized).

Step 5: HTTP module - send to Skipily

Module **HTTP > Make a request**. Method POST. Map the webhook and AI outputs into the fields (Make suggests mappings with *„Map“*, partly via AI):

```
POST https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/products-api
Header:  Content-Type: application/json
```

```
        x-api-key: <YOUR_SKIPILY_PROVIDER_KEY>

{
  "sku":          "{{1.variants[1].sku}}",
  "name":         "{{ cleaned_name }}",
  "manufacturer": "{{1.vendor}}",
  "description":  "{{ cleaned_description }}",
  "price":        {{1.variants[1].price}},
  "currency":     "EUR",
  "ean":          "{{1.variants[1].barcode}}",
  "stock_quantity": {{1.variants[1].inventory_quantity}},
  "in_stock":     {{1.variants[1].inventory_quantity > 0}},
  "weight_kg":    {{1.variants[1].grams / 1000}},
  "images":       ["{{1.image.src}}"],
  "category_id":  {{ skipily_category_id_or_null }},
  "is_active":    true
}
```

Field mapping Shopify → Skipily:

Shopify field	Skipily field (products-api)
title	name (shortened via AI)
vendor	manufacturer
body_html	description (via AI, HTML removed)
variants[].price	price or variants[].price
variants[].sku	sku or variants[].sku
variants[].barcode	ean
variants[].inventory_quantity	stock_quantity / in_stock
variants[].grams	weight_kg (grams/1000)
options + variants	variant_options + variants (1 row per variant)
image.src / images	images[] (first image = main image)
product_type / tags	category_id (via AI suggestion + mapping)

Step 6: Test

- Create/update a product in Shopify → the scenario runs in Make → check the Skipily shop.
- If the product shows up with a clean name/price/stock, it works.

Step 7: Go live

- In Make set **Scheduling** to **ON**. From now on the sync runs automatically on every Shopify change.

4. Variants (diameter, length, colour)

- variant_options = Shopify options (name + values). variants = one row per Shopify variant with its own price/stock/SKU.
- variants replaces ALL existing variants of a product on POST/PUT.

5. Flow B: orders Skipily -> Shopify (optional)

- Second scenario with a Custom webhook. In the Skipily provider profile, set the Make URL as the order webhook.
- Skipily posts signed to that URL on order events. In Make, create a Shopify order (Admin API) or a note/email from it.

6. Good to know

- **New vs. update:** POST creates, PUT ?id=<uuid> updates. For clean updates, store the Skipily uuid per Shopify product in a Make **Data Store**.
- **Stock:** products/update and the Shopify Admin API provide inventory_quantity; optionally add inventory_levels/update.
- **Rate limit:** the Skipily products-api is load-hardened (write/read limit per provider/IP). For bulk import, send in batches.
- **Keep AI costs in check:** use the AI module only in the product flow (not in loops), keep prompts short, cache the result in the Data Store so unchanged products do not call the AI again.
- **Translation:** Skipily translates name/description on demand into 6 languages. The AI in Make does NOT need to translate, only clean up/shorten.